

Complete React.js Notes - From Zero to Hero

Table of Contents

1. [Getting Started with React](#)
 2. [What is JSX?](#)
 3. [Components](#)
 4. [Props](#)
 5. [State Management](#)
 6. [Hooks](#)
 7. [Event Handling](#)
 8. [Conditional Rendering](#)
 9. [Lists and Keys](#)
 10. [Virtual DOM & Reconciliation](#)
 11. [Advanced Topics](#)
-

Getting Started with React {#getting-started}

What is React?

React is a JavaScript library for building user interfaces. It was created by Facebook (Meta) and is used to build single-page applications (SPAs) where the UI updates dynamically without reloading the page.

Key Features:

- **Component-Based:** Build small, reusable pieces of UI
- **Declarative:** You describe what the UI should look like, React handles updates
- **Fast:** Uses Virtual DOM for efficient updates
- **Popular:** Huge community and ecosystem

Setting Up a React Project

Using Vite (Recommended - Fast & Modern):

```
npm create vite@latest my-app  
cd my-app  
npm install  
npm run dev
```

Vite Configuration (vite.config.js):

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    open: true, // Opens browser automatically
  },
});
```

Project Structure:

```
my-app/
├─ node_modules/
├─ public/
├─ src/
│   ├─ App.jsx
│   ├─ main.jsx
│   └─ index.css
├─ index.html
├─ package.json
└─ vite.config.js
```

What is JSX? {#what-is-jsx}

Definition

JSX stands for **JavaScript XML**. It's a syntax extension that allows you to write HTML-like code inside JavaScript.

Important: JSX is NOT HTML inside JavaScript. It's XML-like syntax that looks like HTML but has JavaScript powers.

Example:

```
const element = <h1>Hello, World!</h1>;
```

This looks like HTML, but it's actually JSX that gets converted to JavaScript.

How JSX Works Behind the Scenes

What you write:

```
const element = <h1 className="greeting">Hello!</h1>;
```

What it becomes (after conversion by Vite/Babel):

```
const element = React.createElement("h1", { className: "greeting" }, "Hello!");
```

Final result (React Element - JavaScript Object):

```
{
  type: 'h1',
  props: {
    className: 'greeting',
    children: 'Hello!'
  }
}
```

JSX Rules (Must Follow!)

1. Must Return Single Parent Element

✗ Wrong:

```
return (
  <h1>Title</h1>
  <p>Paragraph</p>
);
```

✓ Correct:

```
return (
  <div>
    <h1>Title</h1>
    <p>Paragraph</p>
  </div>
);
```

✓ Using Fragment (Preferred when you don't need extra div):

```
return (
  <>
    <h1>Title</h1>
    <p>Paragraph</p>
  </>
);
```

2. Use camelCase for Attributes

HTML vs JSX:

```
// HTML
<div class="container" onclick="handleClick()"></div>

// JSX
<div className="container" onClick={handleClick}></div>
```

Common Conversions:

- `class` → `className`
- `for` → `htmlFor`
- `onclick` → `onClick`
- `onchange` → `onChange`
- `tabindex` → `tabIndex`

3. Close All Tags (Even Self-Closing)

✗ Wrong:

```

<br>
<input type="text">
```

✓ Correct:

```

<br />
<input type="text" />
```

4. JavaScript Integration - Use Curly Braces {}

```
const name = "John";
const age = 25;

return (
  <div>
    <h1>Hello {name}!</h1>
    <p>You are {age} years old</p>
    <p>Next year: {age + 1}</p>
  </div>
);
```

5. Styles Must Be JavaScript Objects

✗ Wrong:

```
<div style="color: red; font-size: 20px;"></div>
```

✓ Correct:

```
<div style={{ color: "red", fontSize: "20px" }}></div>
```

Or use a variable:

```
const myStyle = {
  color: "red",
  fontSize: "20px",
  backgroundColor: "yellow",
};

<div style={myStyle}>Styled Text</div>;
```

What You CAN'T Write in JSX

✗ Cannot use these directly inside `{ }`:

1. `if` statements
2. `else` statements
3. `while` loops
4. `for` loops
5. `switch` statements
6. Function declarations

```
// ✗ This will NOT work
return (
  <div>
    {
      if (isLoggedIn) {
        <h1>Welcome!</h1>
      }
    }
  </div>
);
```

What You CAN Write in JSX

✅ You can use:

1. Ternary operator `? :`
2. Logical AND `&&`
3. Logical OR `||`
4. Logical NOT `!`
5. Function calls
6. Array methods (`map`, `filter`, `reduce`)

```
// ✅ Ternary Operator
{
  isLoggedIn ? <h1>Welcome!</h1> : <h1>Please Login</h1>;
}

// ✅ Logical AND
{
  isLoggedIn && <h1>Welcome Back!</h1>;
}

// ✅ Function Call
{
  getName();
}

// ✅ Array Map
{
  users.map((user) => <div key={user.id}>{user.name}</div>);
}
```

JSX Comments

```
{
  /* This is a comment in JSX */
}

return (
  <div>
    {/* Single line comment */}
    <h1>Hello</h1>

    {/*
      Multi-line
      comment
    */}
  </div>
);
```

Components

What is a Component?

A component is a **JavaScript function that returns JSX**. Think of it as a reusable piece of UI.

Simple Example:

```
function Welcome() {
  return <h1>Hello, Welcome to React!</h1>;
}
```

Component Naming Rules

1. Always start with UPPERCASE letter

-  Header, UserProfile, MyButton
-  header, userProfile, mybutton

2. File Naming Convention:

- Contains component: Header.jsx (Capital letter)
- Only elements, no components: utils.js (Small letter)

Types of Components

1. Function Components (Modern & Recommended)

```
function Greeting() {
  return <h1>Hello!</h1>;
}
```

Arrow Function Style:

```
const Greeting = () => {
  return <h1>Hello!</h1>;
};
```

Short syntax (for single element):

```
const Greeting = () => <h1>Hello!</h1>;
```

2. Class Components (Old Style - Less Used Now)

```
import React, { Component } from "react";

class Greeting extends Component {
  render() {
    return <h1>Hello!</h1>;
  }
}
```

Component Composition (Nesting Components)

Components can use other components inside them.

```
// Child Component
const Header = () => {
  return <h1>My Website</h1>;
};

// Child Component
const Footer = () => {
  return <footer>© 2024 My Website</footer>;
};

// Parent Component using child components
const App = () => {
  return (
    <div>
      <Header />
      <main>
        <p>Main Content Here</p>
      </main>
      <Footer />
    </div>
  );
};
```

How to Use Components

main.jsx (Entry Point):

```
import { createRoot } from "react-dom/client";
import App from "./App";

createRoot(document.getElementById("root")).render(<App />);
```

Explanation:

1. `createRoot()` - Creates a root to display React components
2. `getElementById('root')` - Finds the div with id="root" in HTML
3. `.render(<App />)` - Displays the App component inside that div

Using React Elements in Components

```
// React Element (just JSX, not a component)
const heading = <h1>This is an element</h1>;

// Component using the element
const App = () => {
  return (
    <div>
      {heading} {/* Use {} to insert element */}
      <p>More content</p>
    </div>
  );
};
```

StrictMode

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

What is StrictMode?

- A tool to help you write better React code
- Highlights potential problems in your app
- **Only works in development** (not in production)
- Makes components render twice to catch bugs
- **Don't use in production builds**

Benefits:

- Warns about unsafe lifecycle methods
- Warns about deprecated APIs
- Detects unexpected side effects
- Helps with debugging

Props

What are Props?

Props (short for "properties") are how we pass data from parent component to child component. Think of them like function arguments.

Basic Props Example

```
// Child Component
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

// Parent Component
function App() {
  return (
    <div>
      <Greeting name="John" />
      <Greeting name="Sarah" />
      <Greeting name="Mike" />
    </div>
  );
}
```

Props Rules

1. Passing Different Data Types

String (use quotes):

```
<User name="John" />
```

Number, Boolean, Array, Object (use curly braces):

```
<User
  name="John"
  age={25}
  isStudent={true}
  hobbies=["reading", "coding"]
  address={{ city: "New York", country: "USA" }}
/>
```

2. Destructuring Props (Cleaner Code)

Without Destructuring:

```
function User(props) {
  return (
    <div>
      <h1>{props.name}</h1>
      <p>Age: {props.age}</p>
    </div>
  );
}
```

With Destructuring (Preferred):

```
function User({ name, age }) {
  return (
    <div>
      <h1>{name}</h1>
      <p>Age: {age}</p>
    </div>
  );
}
```

3. Default Props

```
function Greeting({ name = "Guest" }) {
  return <h1>Hello, {name}!</h1>;
}

// Usage
<Greeting /> // Shows: Hello, Guest!
<Greeting name="John" /> // Shows: Hello, John!
```

4. Props are Read-Only

✗ Wrong - Cannot modify props:

```
function User({ name }) {
  name = "Changed"; // ✗ ERROR! Cannot change props
  return <h1>{name}</h1>;
}
```

✓ Correct - Use state if you need to change values:

```
function User({ initialName }) {
  const [name, setName] = useState(initialName);

  return (
    <div>
      <h1>{name}</h1>
      <button onClick={() => setName("Changed")}>Change</button>
    </div>
  );
}
```

Children Props

Special prop to pass content between component tags.

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}  
  
// Usage  
<Card>  
  <h1>Title</h1>  
  <p>This is the content</p>  
</Card>;
```

Props vs State

Props	State
Passed from parent	Managed within component
Read-only	Can be changed
External data	Internal data
Like function arguments	Like function variables

State Management {#state-management}

What is State?

State is **data that changes over time** in your component. When state changes, React re-renders the component to show the new data.

Example: Counter app, form inputs, showing/hiding content, etc.

State vs Regular Variables

✗ **Using Regular Variable (Doesn't Work!):**

```
function Counter() {
  let count = 0; // Regular variable

  const increment = () => {
    count = count + 1; // Changes variable
    console.log(count); // Shows updated value in console
    // But UI doesn't update! ❌
  };

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={increment}>Increase</button>
    </div>
  );
}
```

Problem: React doesn't know the variable changed, so it doesn't re-render the UI.

✅ Using State (Correct Way!):

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // State variable

  const increment = () => {
    setCount(count + 1); // Updates state
    // React automatically re-renders with new value! ✅
  };

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={increment}>Increase</button>
    </div>
  );
}
```

Hooks

What are React Hooks?

- React hooks are simple functions that let you add things like state and side effects to your functional components. They help your component remember values, run code when something changes and keep your logic clean.
- A side effect is any work your component does that is not just displaying UI.

Rules of Hooks (Must Follow!)

1. Only Call at Top Level

✗ Wrong:

```
function MyComponent() {
  if (condition) {
    const [count, setCount] = useState(0); // ✗ Inside condition
  }

  for (let i = 0; i < 5; i++) {
    const [name, setName] = useState(""); // ✗ Inside loop
  }

  function helper() {
    const [value, setValue] = useState(0); // ✗ Inside function
  }
}
```

✓ Correct:

```
function MyComponent() {
  // All hooks at the top level
  const [count, setCount] = useState(0);
  const [name, setName] = useState("");
  const [value, setValue] = useState(0);

  // Then your conditions, loops, functions...
  if (condition) {
    // use the state here
  }
}
```

2. Only Call from React Components or Custom Hooks

```
// Inside component
function MyComponent() {
  const [state, setState] = useState(0); // ✓
}

// Inside custom hook
function useCustomHook() {
  const [state, setState] = useState(0); // ✓
}
```

✗ Wrong:

```
// In regular JavaScript function
function regularFunction() {
  const [state, setState] = useState(0); // ✗
}
```

useState Hook

useState is a React hook that lets your component store and update values. A functional component normally cannot remember things, so useState gives it a way to keep track of data between renders.

Why useState exists

A component needs to change data over time, like:

- input values
- counters
- API data
- toggle states

But functions forget values after they run. useState solves this by giving the component a "memory."

What useState does

1. It creates a state value that React keeps safe between renders.
2. It gives you a setter function to update that value.
3. When you change the state, React re-renders the component with the new value.

Syntax:

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

Parts:

- `stateVariable`: Current value of state
- `setStateFunction`: Function to update the state
- `initialValue`: Starting value (can be any type)

Basic Examples

1. Counter:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(count - 1)}>-1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

2. Text Input:

```
function InputExample() {
  const [text, setText] = useState("");

  return (
    <div>
      <input
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <p>You typed: {text}</p>
    </div>
  );
}
```

3. Toggle (Show/Hide):

```
function ToggleExample() {
  const [isVisible, setIsVisible] = useState(false);

  return (
    <div>
      <button onClick={() => setIsVisible(!isVisible)}>Toggle</button>
      {isVisible && <p>Now you see me!</p>}
    </div>
  );
}
```

4. Array State:

```
function TodoList() {
  const [todos, setTodos] = useState(["Task 1", "Task 2"]);

  const addTodo = () => {
    setTodos([...todos, `Task ${todos.length + 1}`]);
  };

  return (
    <div>
      {todos.map((todo, index) => (
        <p key={index}>{todo}</p>
      ))}
      <button onClick={addTodo}>Add Todo</button>
    </div>
  );
}
```


5. Object State:

```
function UserProfile() {
  const [user, setUser] = useState({
    name: "John",
    age: 25,
    email: "john@example.com",
  });

  const updateName = () => {
    setUser({ ...user, name: "Jane" });
  };

  return (
    <div>
      <p>Name: {user.name}</p>
      <p>Age: {user.age}</p>
      <button onClick={updateName}>Change Name</button>
    </div>
  );
}
```

Why useState Triggers Re-render

When you call `setState`:

1. React updates the state value
2. React compares old UI vs new UI (Virtual DOM)
3. React updates only changed parts (Reconciliation)
4. Your component re-renders with new data

State updates are asynchronous:

```
const [count, setCount] = useState(0);

function handleClick() {
  setCount(count + 1);
  console.log(count); // Still shows old value!
  // Because state update is async
}
```

useEffect Hook

- `useEffect` is a React hook that lets you run code after your component renders. It is used for tasks like fetching data, setting timers or updating things outside the UI.
- `useEffect` runs side effects after each render based on the dependencies you provide, so you can control when the effect should run.

What useEffect does

1. It lets you run side effects (fetch API, timers, event listeners, `localStorage`).
2. It runs after the render is complete, so the UI stays smooth.

3. It lets you control when the code runs using the dependency array.

How the dependency array works

- **No array:** runs after every render
- **Empty array []:** runs only on first render (like componentDidMount)
- **[someValue]:** runs when that value changes

Syntax:

```
import { useEffect } from "react";

useEffect(() => {
  // Your side effect code here

  return () => {
    // Cleanup code (optional)
  };
}, [dependencies]);
```

Three Ways to Use useEffect

1. Run After Every Render (No Dependency Array):

```
useEffect(() => {
  console.log("Runs after every render");
});
```

2. Run Only Once (Empty Dependency Array):

```
useEffect(() => {
  console.log("Runs only once after first render");
}, []);
```

Use Case: Data fetching when component loads

3. Run When Specific Values Change:

```
const [count, setCount] = useState(0);

useEffect(() => {
  console.log("Runs when count changes");
}, [count]);
```

Practical Examples

1. Data Fetching:

```
import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Fetch data when component loads
    fetch("https://api.example.com/users")
      .then((res) => res.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  }, []); // Empty array = run once

  if (loading) return <p>Loading...</p>;

  return (
    <div>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}
```

2. Document Title Update:

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]); // Run when count changes

  return (
    <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>
  );
}
```

3. Timer/Interval:

```
function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => {
      setTime(new Date());
    }, 1000);

    // Cleanup function
    return () => {
      clearInterval(timer); // Stop timer when component unmounts
    };
  }, []);

  return <p>Time: {time.toLocaleTimeString()}</p>;
}
```

4. Event Listener:

```
function WindowSize() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => {
      setWidth(window.innerWidth);
    };

    window.addEventListener("resize", handleResize);

    // Cleanup
    return () => {
      window.removeEventListener("resize", handleResize);
    };
  }, []);

  return <p>Window width: {width}px</p>;
}
```

Cleanup Function

The cleanup function runs:

- Before the component is removed from screen
- Before the effect runs again (for dependency changes)

Why Need Cleanup?

- Stop timers
- Cancel API requests
- Remove event listeners
- Prevent memory leaks

```
useEffect(() => {
  // Setup
  const timer = setInterval(() => {
    console.log("Running...");
  }, 1000);

  // Cleanup (important!)
  return () => {
    clearInterval(timer);
  };
}, []);
```

Common Hook Mistakes

✗ Mistake 1: Modifying state directly

```
const [user, setUser] = useState({ name: "John", age: 25 });

// Wrong
user.name = "Jane"; // ✗ Direct mutation

// Correct
setUser({ ...user, name: "Jane" }); // ✓
```

✗ Mistake 2: Using stale state

```
const [count, setCount] = useState(0);

function increment() {
  setCount(count + 1);
  setCount(count + 1); // Won't work as expected
}

// Correct way
function increment() {
  setCount(prev => prev + 1);
  setCount(prev => prev + 1); // ✓ Works correctly
}
```

✗ Mistake 3: Missing dependencies

```
const [count, setCount] = useState(0);

useEffect(() => {
  console.log(count); // Uses count
}, []); // ✗ But count not in dependencies!

// Correct
useEffect(() => {
  console.log(count);
}, [count]); // ✓ Include count
```

Event Handling

What are Events?

Events are actions that happen in the browser (clicks, typing, mouse movements, etc.). React handles these with **event handlers**.

Basic Event Handling

❌ Wrong (HTML way):

```
<button onclick="handleClick()">Click</button>
```

✅ Correct (React way):

```
<button onClick={handleClick}>Click</button>
```

Differences:

1. Use `camelCase` (onClick, not onclick)
2. Pass function reference, not string
3. Don't add `()` when passing function

Why Not Use () ?

❌ Wrong:

```
<button onClick={handleClick()}>Click</button>
```

Problem: Function runs immediately when page loads, not when button is clicked!

✅ Correct:

```
<button onClick={handleClick}>Click</button>
```

Explanation: Pass the function itself. React will call it when button is clicked.

Common Events

```
function EventExamples() {
  return (
    <div>
      {/* Click Events */}
      <button onClick={() => console.log("Clicked!")}>Click Me</button>

      {/* Input Events */}
      <input
        type="text"
        onChange={(e) => console.log(e.target.value)}
        onFocus={() => console.log("Input focused")}
        onBlur={() => console.log("Input lost focus")}
      />

      {/* Form Events */}
      <form onSubmit={(e) => e.preventDefault()}>
        <button type="submit">Submit</button>
      </form>

      {/* Mouse Events */}
      <div
        onMouseEnter={() => console.log("Mouse entered")}
        onMouseLeave={() => console.log("Mouse left")}
        onMouseMove={(e) => console.log(e.clientX, e.clientY)}
      >
        Hover over me
      </div>

      {/* Keyboard Events */}
      <input
        type="text"
        onKeyDown={(e) => console.log("Key down:", e.key)}
        onKeyUp={(e) => console.log("Key up:", e.key)}
      />
    </div>
  );
}
```

Event Handler Patterns

1. Inline Function:

```
<button onClick={() => console.log("Clicked")}>Click</button>
```

2. Separate Function:

```
function MyComponent() {
  const handleClick = () => {
    console.log("Clicked");
  };

  return <button onClick={handleClick}>Click</button>;
}
```

3. Passing Arguments:

```
function MyComponent() {
  const handleClick = (name) => {
    console.log("Hello", name);
  };

  return (
    <div>
      <button onClick={() => handleClick("John")}>Click</button>
    </div>
  );
}
```

Synthetic Events

What is Synthetic Event?

A Synthetic event is a cross browser wrapper around the browser's native event object. React creates this wrapper to ensure consistent behavior across different browsers.

React wraps browser's native events into its own event system called **Synthetic Events**. This ensures events work the same way across all browsers.

```
function handleClick(event) {
  console.log(event); // This is Synthetic Event
  console.log(event.nativeEvent); // This is browser's native event
}
```

Benefits:

- Cross-browser compatibility
- Better performance
- Automatic cleanup

Common Event Properties:

```
function handleEvent(e) {
  e.preventDefault(); // Prevent default behavior
  e.stopPropagation(); // Stop event bubbling

  console.log(e.target); // Element that triggered event
  console.log(e.currentTarget); // Element event handler is attached to
  console.log(e.type); // Event type (click, change, etc.)
}
```


Form Handling Example

```
import { useState } from "react";

function LoginForm() {
  const [formData, setFormData] = useState({
    email: "",
    password: "",
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({
      ...formData,
      [name]: value,
    });
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page reload
    console.log("Form submitted:", formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="Email"
      />
      <input
        type="password"
        name="password"
        value={formData.password}
        onChange={handleChange}
        placeholder="Password"
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

Conditional Rendering

Conditional rendering means showing different UI based on certain conditions. It is a way to show or hide parts of your component depending on some value or state.

Examples

- If the user is logged in, show the dashboard.
- If not, show the login button.
- If data is loading, show a loader.
- If data is ready, show the content.

How you do it in React

You can use:

- if statements
- ternary operator
- logical AND (&&)

So conditional rendering is just choosing which UI to display depending on the situation.

Showing different content based on conditions (like if-else in normal JavaScript).

Methods for Conditional Rendering

1. Ternary Operator (?:)

Syntax: `condition ? trueValue : falseValue`

```
function Greeting({ isLoggedIn }) {
  return (
    <div>{isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please sign in</h1>}</div>
  );
}
```

With multiple elements:

```
{
  isLoggedIn ? (
    <>
      <h1>Welcome!</h1>
      <button>Logout</button>
    </>
  ) : (
    <>
      <h1>Hello Guest</h1>
      <button>Login</button>
    </>
  );
}
```

2. Logical AND (&&)

Use when you want to show something only if condition is true.

```
function Notification({ hasMessage }) {
  return <div>{hasMessage && <p>You have new messages!</p>}</div>;
}
```

Examples:

```
// Show element if true
{
  isLoggedIn && <UserDashboard />;
}

// Show element if count > 0
{
  count > 0 && <p>Count: {count}</p>;
}

// Show element if array has items
{
  items.length > 0 && <ItemList items={items} />;
}
```

⚠ Warning: Be careful with numbers

```
{
  count && <p>Count: {count}</p>;
}
```

Problem: If count is 0, it will show 0 instead of nothing!

✅ Better:

```
{
  count > 0 && <p>Count: {count}</p>;
}
```

3. Logical OR (||)

Show default value if first value is falsy.

```
function UserName({ name }) {
  return <h1>Hello, {name || "Guest"}!</h1>;
}

// Usage
<UserName name="John" /> // Shows: Hello, John!
<UserName /> // Shows: Hello, Guest!
```

4. Logical NOT (!)

Invert a boolean value.

```
function ToggleButton({ isOn, onToggle }) {
  return (
    <div>
      <p>Status: {isOn ? "ON" : "OFF"}</p>
      {!isOn && <p>Please turn on to continue</p>}
      <button onClick={onToggle}>{isOn ? "Turn OFF" : "Turn ON"}</button>
    </div>
  );
}
```

5. If-Else Outside JSX

```
function UserStatus({ isLoggedIn, isPremium }) {
  // Use if-else before return
  if (!isLoggedIn) {
    return <h1>Please login</h1>;
  }

  if (isPremium) {
    return <h1>Welcome Premium User!</h1>;
  }

  return <h1>Welcome Free User!</h1>;
}
```

6. Switch Case (Outside JSX)

```
function OrderStatus({ status }) {
  let message;

  switch (status) {
    case "pending":
      message = "Order is being processed";
      break;
    case "shipped":
      message = "Order is on the way";
      break;
    case "delivered":
      message = "Order delivered successfully";
      break;
    default:
      message = "Unknown status";
  }

  return <p>{message}</p>;
}
```

7. Immediately Invoked Function Expression (IIFE)

```
function ComplexCondition({ status }) {
  return (
    <div>
      {(() => {
        if (status === "loading") {
          return <p>Loading...</p>;
        } else if (status === "error") {
          return <p>Error occurred!</p>;
        } else {
          return <p>Success!</p>;
        }
      })()}
    </div>
  );
}
```

Multiple Conditions Example

```
function Dashboard({ user, loading, error }) {
  // Loading state
  if (loading) {
    return <div>Loading...</div>;
  }

  // Error state
  if (error) {
    return <div>Error: {error}</div>;
  }

  // No user
  if (!user) {
    return <div>Please login</div>;
  }

  // Success state
  return (
    <div>
      <h1>Welcome, {user.name}!</h1>
      {user.isPremium && <span>★ Premium Member</span>}
      {user.unreadMessages > 0 && (
        <p>You have {user.unreadMessages} new messages</p>
      )}
    </div>
  );
}
```

Lists and Keys

Rendering Lists

The .map() Method

Use `.map()` to transform arrays into JSX elements.

Basic Example:

```
function FruitList() {
  const fruits = ["Apple", "Banana", "Orange"];

  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

With Objects:

```
function UserList() {
  const users = [
    { id: 1, name: "John", age: 25 },
    { id: 2, name: "Sarah", age: 30 },
    { id: 3, name: "Mike", age: 22 },
  ];

  return (
    <div>
      {users.map((user) => (
        <div key={user.id}>
          <h2>{user.name}</h2>
          <p>Age: {user.age}</p>
        </div>
      ))}
    </div>
  );
}
```

Why We Need Keys

Keys help React identify which items changed, added, or removed.

✗ Without Key (Warning in Console):

```
{
  items.map((item) => <li>{item}</li>);
}
```

✓ With Key:

```
{
  items.map((item) => <li key={item.id}>{item}</li>);
}
```

Key Rules

1. **Keys must be unique among siblings**
2. **Keys should be stable (don't change)**
3. **Keys should be predictable**

✗ Bad Key Choices:

```
// Using index (only OK if list never changes)
{
  items.map((item, index) => <li key={index}>{item}</li>);
}

// Using random number (changes every render!)
{
  items.map((item) => <li key={Math.random()}>{item}</li>);
}
```

✓ Good Key Choices:

```
// Using unique id from data
{
  items.map((item) => <li key={item.id}>{item}</li>);
}

// Using unique property
{
  users.map((user) => <div key={user.email}>{user.name}</div>);
}
```

When to Use Index as Key

✓ OK to use index when:

- List items don't have unique IDs
- List never reorders
- List never filters
- List is static

✗ Don't use index when:

- Items can be reordered
- Items can be added/removed from middle
- Items can be filtered

Filter and Map Together

```
function ProductList({ products, category }) {
  return (
    <div>
      {products
        .filter((product) => product.category === category)
        .map((product) => (
          <div key={product.id}>
            <h3>{product.name}</h3>
            <p>${product.price}</p>
          </div>
        ))}
    </div>
  );
}
```

Conditional Rendering in Lists

```
function TaskList({ tasks }) {
  return (
    <ul>
      {tasks.map((task) => (
        <li
          key={task.id}
          style={{
            textDecoration: task.completed ? "line-through" : "none",
          }}
        >
          {task.title}
          {task.important && <span> ⚠ </span>}
        </li>
      ))}
    </ul>
  );
}
```


Empty List Handling

```
function TodoList({ todos }) {
  if (todos.length === 0) {
    return <p>No todos yet. Add one!</p>;
  }

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}

// Or inline
function TodoList({ todos }) {
  return (
    <div>
      {todos.length === 0 ? (
        <p>No todos yet!</p>
      ) : (
        <ul>
          {todos.map((todo) => (
            <li key={todo.id}>{todo.text}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

Virtual DOM & Reconciliation

What is the DOM?

DOM (Document Object Model) is how browsers represent web pages. It's a tree structure of HTML elements.

Example HTML:

```
<div id="root">
  <h1>Hello</h1>
  <p>World</p>
</div>
```

DOM Tree:

```
div#root
├─ h1 ("Hello")
└─ p ("World")
```

Problem with Real DOM:

- Updating DOM is SLOW
- Every small change = whole tree update
- Inefficient for dynamic apps

What is Virtual DOM?

Virtual DOM is a lightweight JavaScript copy of the real DOM. It's just a JavaScript object.

Example Virtual DOM Object:

```
{
  type: 'div',
  props: { id: 'root' },
  children: [
    { type: 'h1', props: {}, children: ['Hello'] },
    { type: 'p', props: {}, children: ['World'] }
  ]
}
```

How Virtual DOM Works

Step 1: Initial Render

React creates Virtual DOM → Converts to Real DOM → Shows on screen

Step 2: State Changes

1. State updates
2. React creates NEW Virtual DOM
3. Compares OLD Virtual DOM vs NEW Virtual DOM (Diffing)
4. Finds differences
5. Updates ONLY changed parts in Real DOM

Reconciliation Process

Reconciliation is React's algorithm to compare Virtual DOMs and update the real DOM efficiently.

Example:

```
// Before state change
<div>
  <h1>Count: 0</h1>
  <p>Static text</p>
</div>

// After state change (count = 1)
<div>
  <h1>Count: 1</h1>
  <p>Static text</p>
</div>
```

What React Does:

1. Compares old vs new Virtual DOM
2. Finds only `<h1>` changed
3. Updates only the text inside `<h1>` in real DOM
4. Leaves `<p>` untouched (no re-render!)

Diffing Algorithm

React uses smart rules to compare trees efficiently:

1. Different Element Types → Replace

```
// Old
<div>Hello</div>

// New
<span>Hello</span>

// React: Destroy <div>, create new <span>
```

2. Same Element Type → Update Props

```
// Old
<div className="before">Hello</div>

// New
<div className="after">Hello</div>

// React: Only update className, keep element
```

3. Component Elements → Re-render

```
// Old
<MyComponent name="John" />

// New
<MyComponent name="Jane" />

// React: Re-render MyComponent with new props
```

4. Keys in Lists → Match Correctly

```
// Old
<ul>
  <li key="1">First</li>
  <li key="2">Second</li>
</ul>

// New
<ul>
  <li key="1">First</li>
  <li key="2">Second</li>
  <li key="3">Third</li>
</ul>

// React: Keep first two, add third (efficient!)
```

Why React is Fast

Traditional Way (Without Virtual DOM):

Change → Update entire page → Slow

React Way (With Virtual DOM):

Change → Update Virtual DOM (fast)
 → Find differences (fast)
 → Update only changes in Real DOM (fast)

Performance Benefits:

- Batch updates (combines multiple changes)
- Minimal DOM operations
- Smart diffing algorithm
- Efficient re-renders

React Fiber (Advanced)

React Fiber is React's reimplementation of the reconciliation algorithm (React 16+).

Features:

- Can pause and resume work
 - Prioritizes updates (user interactions > background tasks)
 - Better performance for complex apps
-

Advanced Topics {#advanced-topics}

1. Config-Driven UI

Building UI dynamically based on configuration data (usually JSON from backend) instead of hardcoding everything.

Config driven UI means building your interface based on a configuration object instead of writing the UI manually. The UI changes based on data, not hard-coded components.

Simple Meaning

You describe **what** to show in a config (JSON, array, object), and the UI is generated from that config.

Why it's useful

- Easy to update UI without changing code
- Reusable for multiple screens
- Good for forms, tables, menus and dashboards
- Reduces repeated code

Example Idea

You create a config like:

```
const formConfig = [  
  { type: "text", label: "Name" },  
  { type: "email", label: "Email" },  
  { type: "number", label: "Age" },  
];
```

Benefits:

- Change UI without code deployment
- Different UI for different users/regions
- Easy A/B testing
- Centralized control

Example:

```
// Config from backend
const dashboardConfig = {
  title: "User Dashboard",
  widgets: [
    { id: 1, type: "chart", title: "Sales", data: [...] },
    { id: 2, type: "table", title: "Orders", columns: [...] },
    { id: 3, type: "card", title: "Revenue", value: 5000 }
  ]
};

// Component renders based on config
function Dashboard({ config }) {
  return (
    <div>
      <h1>{config.title}</h1>
      {config.widgets.map(widget => {
        switch(widget.type) {
          case 'chart':
            return <ChartWidget key={widget.id} {...widget} />;
          case 'table':
            return <TableWidget key={widget.id} {...widget} />;
          case 'card':
            return <CardWidget key={widget.id} {...widget} />;
          default:
            return null;
        }
      })}
    </div>
  );
}
```

2. Shimmer UI (Loading Skeleton)

Shimmer UI is a loading placeholder that looks like a light animation moving across gray blocks. It shows users that the content is loading instead of leaving an empty screen.

Simple Meaning

It is a fake skeleton layout that appears before real data arrives. It helps users feel the app is fast and prevents a blank screen.

Why it's used

- Makes loading feel smoother
- Gives structure of the page before content loads
- Better user experience than a spinner

Example Use Case

When fetching a list of products, you first show gray boxes with a shimmer effect. Once data arrives, the real product cards replace those boxes.

Short Summary

Shimmer UI is a skeleton loading screen with a moving light effect that shows while the real data is being fetched.

Show placeholder while content loads.

```

function ShimmerCard() {
  return (
    <div className="shimmer-card">
      <div
        className="shimmer-line"
        style={{ width: "100%", height: "200px" }}
      ></div>
      <div
        className="shimmer-line"
        style={{ width: "80%", height: "20px" }}
      ></div>
      <div
        className="shimmer-line"
        style={{ width: "60%", height: "20px" }}
      ></div>
    </div>
  );
}

// CSS for shimmer effect
/*
.shimmer-line {
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
}

@keyframes shimmer {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}
*/

function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/products")
      .then((res) => res.json())
      .then((data) => {
        setProducts(data);
        setLoading(false);
      });
  }, []);

  if (loading) {
    return (
      <div>
        <ShimmerCard />
        <ShimmerCard />
        <ShimmerCard />
      </div>
    );
  }

  return (
    <div>
      {products.map((product) => (
        <ProductCard key={product.id} {...product} />
      ))}
    </div>
  );
}

```

Important Concepts Summary

CORS (Cross-Origin Resource Sharing)

Security feature that controls which websites can access your backend API.

CORS is a security rule in browsers that controls which websites are allowed to request data from another domain.

Simple Meaning

If your frontend and backend are on different domains, the browser blocks the request unless the server says it's allowed.

Why it exists

To stop websites from accessing data from another site without permission.

How it works

The server must send a header like:

```
Access-Control-Allow-Origin: \*
```

or a specific domain:

```
Access-Control-Allow-Origin: [https://your-site.com](https://your-site.com)
```

Short Summary

CORS is a browser security system that allows or blocks requests made from one website to another.

Problem:

```
Frontend: http://localhost:3000  
Backend: http://localhost:5000  
  
Browser blocks request! (Different origins)
```

Solution: Backend must allow requests from your frontend.

```
// Express.js backend example  
app.use((req, res, next) => {  
  res.header("Access-Control-Allow-Origin", "http://localhost:3000");  
  res.header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE");  
  res.header("Access-Control-Allow-Headers", "Content-Type");  
  next();  
});
```


Axios vs Fetch

What is Axios?

Axios is a JavaScript library used to make HTTP requests from the browser or Node.js. It helps you send data to a server and get data back in a simple and clean way.

Why developers use Axios

- Easy to use
 - Handles JSON automatically
 - Better error handling
 - Works in all browsers
 - Supports request cancellation, interceptors and timeouts
-

Axios vs Fetch: Key Differences

1. Syntax

Axios is shorter and cleaner:

```
axios.get("/api/user");
```

Fetch needs more steps:

```
fetch("/api/user").then((res) => res.json());
```

2. JSON Handling

- Axios converts JSON automatically.
- Fetch requires `res.json()` every time.

3. Error Handling

- Axios treats non-200 responses as errors.
- Fetch only errors on network issues, not on 4xx or 5xx responses.

4. Browser Support

- Axios works in older browsers.
- Fetch is not fully supported in older browsers without polyfills.

5. Request Cancellation

- Axios supports cancelling requests.
- Fetch needs extra code (AbortController).

6. Interceptors

- Axios has built-in interceptors for adding tokens, logging or retry logic.
- Fetch does not have this built in.

Short Summary

Axios is a simpler, more feature-rich HTTP library. Fetch is built-in but needs extra handling for JSON, errors and features like cancellation.

Fetch (Built-in):

```
fetch("https://api.example.com/data")
  .then((res) => res.json())
  .then((data) => console.log(data))
  .catch((err) => console.error(err));
```

Axios (Library):

```
import axios from "axios";

axios
  .get("https://api.example.com/data")
  .then((res) => console.log(res.data))
  .catch((err) => console.error(err));
```

Axios Benefits:

- Automatic JSON parsing
 - Better error handling
 - Request/response interceptors
 - Cancel requests
 - Timeout support
-